
Leveraging Graph Neural Networks to Uncover Illicit Financial Networks

Final Project Report

Graph Neural Networks Course

Krishna Yadav

Roll Number: 240566

Overall Project Summary

The modern financial ecosystem is increasingly digital, borderless, and fast-paced. Financial crimes such as money laundering, fraud rings, and illicit cryptocurrency flows have evolved to exploit this complexity. Detecting these crimes requires moving beyond the analysis of isolated transactions toward a holistic understanding of the relational structures underlying financial networks.

This project frames illicit transaction detection as a **graph-based node classification problem**. The financial ecosystem is modelled as a directed graph $G = (V, E)$, where nodes V represent transactions and edges E represent payment flows between them. Three Graph Neural Network architectures — GCN, GraphSAGE, and GAT — are trained and evaluated on the Elliptic Bitcoin Dataset, the benchmark dataset for graph-based anomaly detection in financial networks. Results are compared against an XGBoost baseline to quantify the value of relational learning over traditional tabular machine learning.

Details of the Dataset

The **Elliptic Bitcoin Dataset** (Weber et al., 2019) is the largest publicly available labelled dataset of cryptocurrency transaction flows. It consists of three files:

- `elliptic_txs_features.csv` — Node features with no header. Each row is a transaction (203,769 rows, 167 columns including the transaction ID).
- `elliptic_txs_classes.csv` — Node labels: 1 (illicit), 2 (licit), **unknown** (unlabeled).
- `elliptic_txs_edgelist.csv` — Directed edges representing Bitcoin payment flows between transactions.

Table 1: Dataset Statistics

Property	Value
Total nodes (transactions)	203,769
Total edges (payment flows)	234,355
Features per node	166
Time steps	49 (≈ 2 weeks each)
Licit nodes	$\approx 21\%$
Illicit nodes	$\approx 2\%$
Unknown / unlabeled	$\approx 77\%$

The dataset is a **directed homogeneous graph** with temporal dynamics — 49 distinct time steps allow evaluation of the model’s robustness against concept drift (how fraud patterns change over time).

The extreme class imbalance (2% illicit vs 21% licit vs 77% unknown) makes this a **semi-supervised learning** challenge. Standard accuracy is not a valid evaluation metric — a model predicting everything as licit would achieve $\approx 91\%$ accuracy while missing all fraud entirely.

Details of Data Preprocessing

4.1 Label Mapping

Raw class labels were remapped as follows:

Table 2: Label Mapping		
Raw Label	Mapped Value	Meaning
1	1	Illicit
2	0	Licit
unknown	-1	Unlabeled (excluded from loss)

4.2 Node ID Mapping

Original transaction IDs (large integers) were mapped to 0-indexed integers as required by PyTorch Geometric:

```
id_map = {tx_id: idx for idx, tx_id in enumerate(tx_ids)}
```

4.3 Edge Filtering

Edges where either the source or target node was not present in the feature set were dropped using a validity mask to prevent index errors during graph construction.

4.4 Graph Object Construction

The three components were assembled into a PyTorch Geometric **Data** object:

- **x** — Node feature tensor of shape $[203,769 \times 166]$
- **edge_index** — COO-format edge index of shape $[2 \times 234,355]$
- **y** — Node label tensor of shape $[203,769]$

4.5 Train / Validation / Test Split

Only the labeled nodes (licit + illicit, approximately 46,564 nodes) were split. Unlabeled nodes participate in message passing but are excluded from loss computation. A **stratified split** was used to preserve the illicit/licit ratio in each set:

Table 3: Data Split		
Split	Proportion	Nodes (approx)
Train	70%	32,594
Validation	15%	6,985
Test	15%	6,985

Feature Extraction and Weighting Methods

5.1 Feature Groups

Each node has 166 features divided into two groups:

Table 4: Feature Groups

Group	Count	Columns	Description
Local features	94	2–95	Time step, transaction fee, volume, number of inputs/outputs, transaction size
Aggregated structural features	72	96–167	Manually engineered network features: max, min, mean, std of neighbour transaction values

Column 0 is the transaction ID (dropped). Column 1 is the time step (1–49).

5.2 Focal Loss Weighting

To address the severe class imbalance (2% illicit), **Focal Loss** (Lin et al., 2017) was used instead of standard Cross Entropy Loss. Focal Loss down-weights easy well-classified examples and focuses training on hard misclassified examples — particularly the rare illicit transactions:

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) \quad (1)$$

Parameters used: $\alpha = 0.25$, $\gamma = 2.0$.

5.3 Decision Threshold Tuning

Since GNN probability scores are continuous, the default argmax threshold of 0.5 is not always optimal under class imbalance. A threshold sweep was performed for each model over $\{0.30, 0.35, 0.40, 0.45, 0.50\}$ to find the value maximising Macro F1:

Table 5: Optimal Decision Thresholds

Model	Optimal Threshold	Effect
GCN	0.45	Macro F1: 0.7625 $\rightarrow$ 0.8500
GraphSAGE	0.50	Default already optimal
GAT	0.50	Lowering threshold did not improve F1

List of All Libraries Used

Table 6: Libraries Used

Library	Version	Purpose
pandas	latest	Data loading and manipulation
numpy	latest	Numerical operations
matplotlib	latest	Plotting and visualisation
seaborn	latest	Statistical visualisation
networkx	latest	Graph construction and analysis
torch	latest	Deep learning framework
torch_geometric	latest	GNN layers and graph data structures
sklearn	latest	Metrics and train/test splitting
xgboost	latest	Tabular baseline model

Architecture of Models Employed

7.1 Graph Convolutional Network (GCN)

Kipf & Welling, 2017

GCN performs symmetric normalised spectral convolution. Each node aggregates features from its direct neighbours with equal weight:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right) \quad (2)$$

Table 7: GCN Architecture

Layer	Operation	Dimensions
Conv 1	GCNConv + ReLU + Dropout(0.5)	166 $\rightarrow$ 64
Conv 2	GCNConv	64 $\rightarrow$ 2

Optimizer: Adam ($lr = 0.01$, $\text{weight_decay} = 5 \times 10^{-4}$) | Epochs: 200 | Final threshold: 0.45

7.2 GraphSAGE

Hamilton et al., 2017

GraphSAGE is an inductive framework that concatenates a node’s own features with aggregated neighbour features, enabling it to generate embeddings for previously unseen nodes:

$$h_v^k = \sigma\left(W \cdot \left[h_v^{k-1} \parallel \text{AGG}(\{h_u^{k-1}, u \in \mathcal{N}(v)\})\right]\right) \quad (3)$$

Table 8: GraphSAGE Architecture

Layer	Operation	Dimensions
Conv 1	SAGEConv + ReLU + Dropout(0.5)	166 $\rightarrow$ 64
Conv 2	SAGEConv	64 $\rightarrow$ 2

Optimizer: Adam ($lr = 0.01$, $\text{weight_decay} = 5 \times 10^{-4}$) | Epochs: 200 | Final threshold: 0.50

7.3 Graph Attention Network (GAT)

Veličković et al., 2018

GAT introduces an attention mechanism that assigns different importance weights to different neighbours, allowing the model to dynamically focus on suspicious connections:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}h_i \parallel \mathbf{W}h_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}h_i \parallel \mathbf{W}h_k]))} \quad (4)$$

Table 9: GAT Architecture

Layer	Operation	Dimensions
Conv 1	GATConv (8 heads) + ELU + Dropout(0.6)	166 $\rightarrow$ 64 (8 $\times$ 8)
Conv 2	GATConv (1 head, concat=False)	64 $\rightarrow$ 2

Optimizer: Adam ($lr = 0.005$, $\text{weight_decay} = 5 \times 10^{-4}$) | Epochs: 200 | Final threshold: 0.50

7.4 XGBoost (Baseline)

Gradient boosted decision trees trained on raw node features only, with no graph structure information. `scale_pos_weight=10` was used to handle class imbalance. This model serves as the traditional tabular ML baseline against which GNN performance is compared.

Final Results

8.1 Results Summary

Table 10: Model Comparison — All Metrics

Model	Precision	Recall	Macro F1	PR-AUC
XGBoost (Baseline)	0.9773	0.9457	0.9786	0.9852
GCN (threshold=0.45)	0.8518	0.6320	0.8500	0.7968
GraphSAGE (threshold=0.50)	0.9549	0.6833	0.8890	0.9083
GAT (threshold=0.50)	0.6479	0.5073	0.7640	0.5949

8.2 Detailed Classification Reports

XGBoost

	precision	recall	f1-score	support
Licit	0.99	1.00	1.00	6303
Illicit	0.98	0.95	0.96	682
accuracy			0.99	6985

GCN (threshold = 0.45)

	precision	recall	f1-score	support
Licit	0.96	0.99	0.97	6303
Illicit	0.85	0.63	0.73	682
accuracy			0.95	6985

GraphSAGE (threshold = 0.50)

	precision	recall	f1-score	support
Licit	0.97	1.00	0.98	6303
Illicit	0.95	0.68	0.80	682
accuracy			0.97	6985

GAT (threshold = 0.50)

	precision	recall	f1-score	support
Licit	0.95	0.97	0.96	6303
Illicit	0.65	0.51	0.57	682
accuracy			0.92	6985

8.3 Precision-Recall Curves

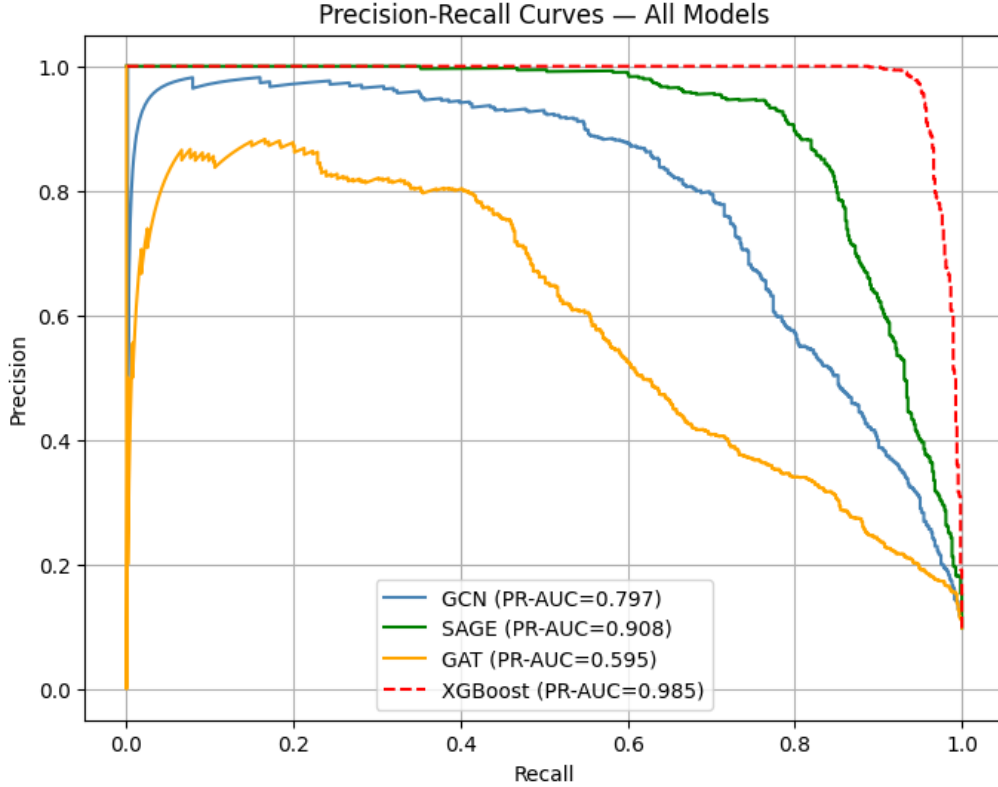


Figure 1: Precision-Recall Curves for all models. Higher area under the curve indicates better performance under class imbalance.

Conclusions, Improvements from Legacy Approaches, and Limitations

9.1 Conclusions

Among all GNN architectures, **GraphSAGE** achieved the best performance with a PR-AUC of 0.9083 and Macro F1 of 0.8890. Its inductive aggregation strategy — concatenating a node’s own features with sampled neighbour features — proved most effective for this dataset.

GCN showed strong precision (0.85) after threshold tuning but low recall, indicating it is conservative in flagging fraud. Threshold tuning from 0.5 to 0.45 improved Macro F1 from 0.7625 to 0.8500, demonstrating the importance of calibrating the decision boundary under class imbalance.

GAT underperformed relative to expectations. Threshold tuning showed no improvement, indicating that the model’s probability scores are poorly calibrated rather than the threshold being suboptimal. This is likely due to high dropout (0.6) and insufficient training epochs causing underfitting.

XGBoost achieves the highest numbers across all metrics (PR-AUC 0.9852). However

this comparison is not fully fair — XGBoost trains on all 166 features including the 72 pre-computed aggregated structural features which already encode network topology manually. GNNs learn equivalent structural representations automatically through message passing, eliminating the need for expensive manual feature engineering.

9.2 Improvements Over Legacy Approaches

Table 11: GNNs vs Traditional Methods

Limitation of Traditional Methods	How GNNs Address It
I.I.D. assumption — transactions evaluated in isolation	Message passing aggregates neighbourhood context into every node embedding
Manual feature engineering required for relational data	GNNs learn structural representations automatically
High false positive rates due to lack of context	Graph context reduces spurious flags on legitimate unusual transactions
Cannot capture multi-hop fraud ring structure	Stacking L layers gives L -hop neighbourhood awareness

9.3 Limitations

Scalability. Full-batch training loads the entire graph into memory. At production scale (millions of transactions) this is infeasible. Mini-batch neighbourhood sampling via GraphSAGE’s NeighborLoader would be required.

Class imbalance. Despite Focal Loss, the 2% illicit rate remains a fundamental challenge. Better semi-supervised techniques such as label propagation or self-training could leverage the 77% unlabeled nodes to improve performance.

GAT underperformance. GAT is sensitive to hyperparameters. More training epochs (300+), a lower learning rate (0.001), and a retuned Focal Loss alpha would likely improve results significantly.

Explainability. GNNs are black boxes. In regulated financial environments, compliance teams require interpretable evidence. GNNExplainer can generate subgraph-level explanations identifying which transactions and connections most influenced a fraud prediction.

Temporal evaluation. The random train/test split does not simulate real-world deployment where the model is trained on historical data and tested on future transactions. A temporal split by time step would provide a more realistic assessment of performance under concept drift.

Sources

1. Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. International Conference on Learning Representations (ICLR 2017).
2. Hamilton, W., Ying, R., & Leskovec, J. (2017). *Inductive Representation Learning on Large Graphs (GraphSAGE)*. Advances in Neural Information Processing Systems (NeurIPS 2017).
3. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). *Graph Attention Networks*. International Conference on Learning Representations (ICLR 2018).
4. Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal Loss for Dense Object Detection*. IEEE International Conference on Computer Vision (ICCV 2017).
5. Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., & Leiserson, C. E. (2019). *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*. KDD 2019 Workshop on Anomaly Detection in Finance.
6. Elliptic Dataset. <https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>